6

Imbalanced Datasets and Data Sampling

April, 2026

Rachid BENOUINI

Why Does Imbalance Occur in Real-World Data ?

Most critical events are rare and inherently imbalanced

- **Fraud Detection:** **Most transactions are normal**, fraud is rare.
- **Medical Diagnosis:** **Diseases affect a small portion** of the population.
- **Cybersecurity:** **Attacks occur less frequently** than normal activity.

Critical events are minority classes and are naturally underrepresented in real-world data.

It's crucial to detect and handle the class imbalance

- **Manufacturing Defects**: Most products are fine, but **defective ones must be found**.
- **Medical Imaging**: Most scans are normal, but **detecting early cancer is critical.**
- **Autonomous Vehicles**: Most roads are safe, but **detecting rare obstacles is life-saving.**

Minority class misclassification or misdetection has serious consequences.

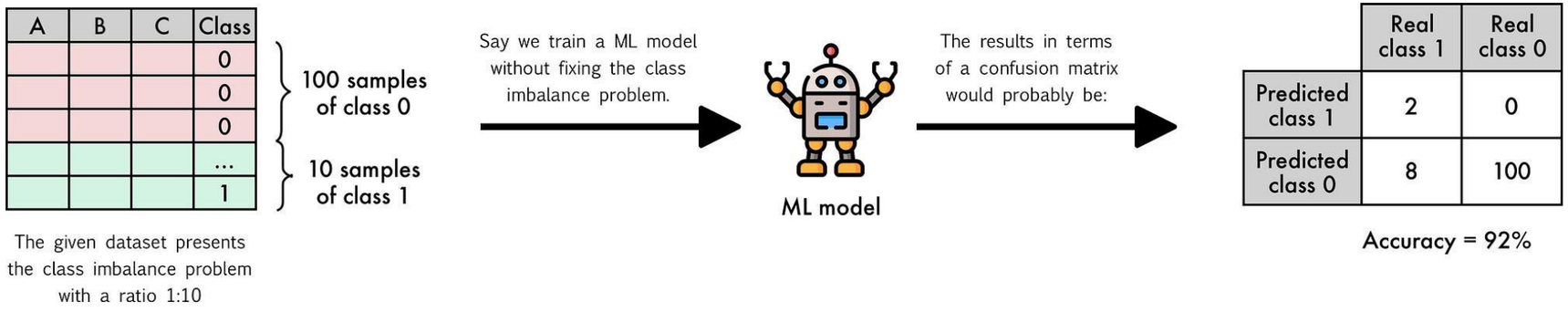
Definitions

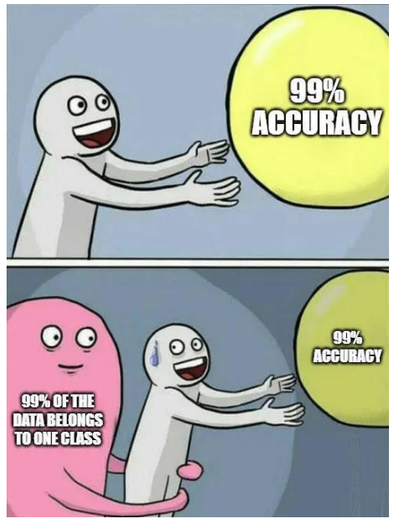
Class imbalance

- The problem of class imbalance arises when the samples for each class are unbalanced, that is, there is no balanced ratio between the distribution of the classes.
- This imbalance can be slight or strong.

The accuracy paradox

- Accuracy is commonly used as the de facto metric, however for the class imbalance problem it would not be a good option since accuracy could be misleading.

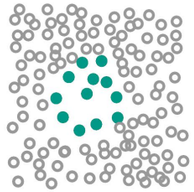




Even if we use other metrics, like **F1-Score**, the model still overfit on the majority class.

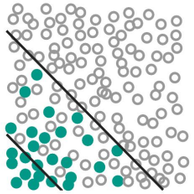
Impact of Imbalanced Data on Machine Learning

Most ML classifiers, from traditional **ML** to advanced **Neural Netwroks**, assume balance class distribution.

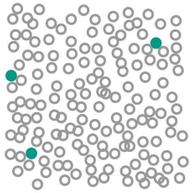


Lack of Density in the Training Dataset: The dataset may have too few samples for certain classes, making it hard for the model to learn generalizable patterns.

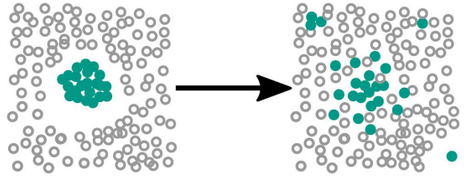
Small Disjuncts: If a class has only a few, widely separated instances, the model may struggle to recognize all variations of that class.



Class Overlapping: If the boundary between classes is not clear, a classifier may misclassify instances, leading to lower performance.



Minority Class Identification as Noise: Noisy data can mislead the model into learning incorrect patterns, reducing accuracy and robustness.



Minority Class Data Shift: If the model is trained on data with one distribution but tested on data from another, its performance may degrade significantly.

It's all about ***learning strutural*** or ***sementic patterns***.

Taxonomy of Solutions of Imbalance Issue

Solutions of Imbalance Issue

Data-Level

Under-Sampling, Over-sampling & Hybrid

- Random Over-sampling
- SMOTE
- SMOTE-ENC
- BorderlineSMOTE
- SVMSMOTE
- ADASYN
- Random Under-sampling
- Cluster Centroids
- Tomek Links
- NearMiss

Algorithm-Level

Cost-Sensitive Learning & Adjustments

- Class Weighting
- Cost-Sensitive Learning
- Decision Threshold Adjustment
- Anomaly Detection-Based Models
- Weighted Loss Function

Ensemble-Based Methods

Boosting & Bagging Techniques

- Adaptive Boosting
- Balanced Random Forest Classifier
- SMOTEBoost
- Ensemble WMV
- Ensemble WELM
- RUSBoost Classifier
- EasyEnsemble Classifier
- BalanceCascade

Evaluation-Level

Metrics & Validation Strategies

- Precision & Recall
- F1-Score
- Confusion Matrix
- Stratified Cross-Validation
- Weighted Majority Voting (WMV)
- Balanced Error Rate (BER)
- Geometric mean (Gmean)
- Matthews Corr. Coef. (MCC)
- AUC-ROC

Imbalanced Learning Frameworks and Libraries

https://imbalanced-ensemble.readthedocs.io/en/latest/

- `imbalanced-ensemble` is a Python toolbox for quick implementing and deploying ensemble learning.
- Includes more than 15 ensemble algorithms based on re-sampling and cost-sensitive learning (e.g., SMOTEBoost/Bagging, RUSBoost/Bagging, AdaCost, EasyEnsemble, BalanceCascade, SelfPacedEnsemble, ...).
- `pip install imbalanced-ensemble`

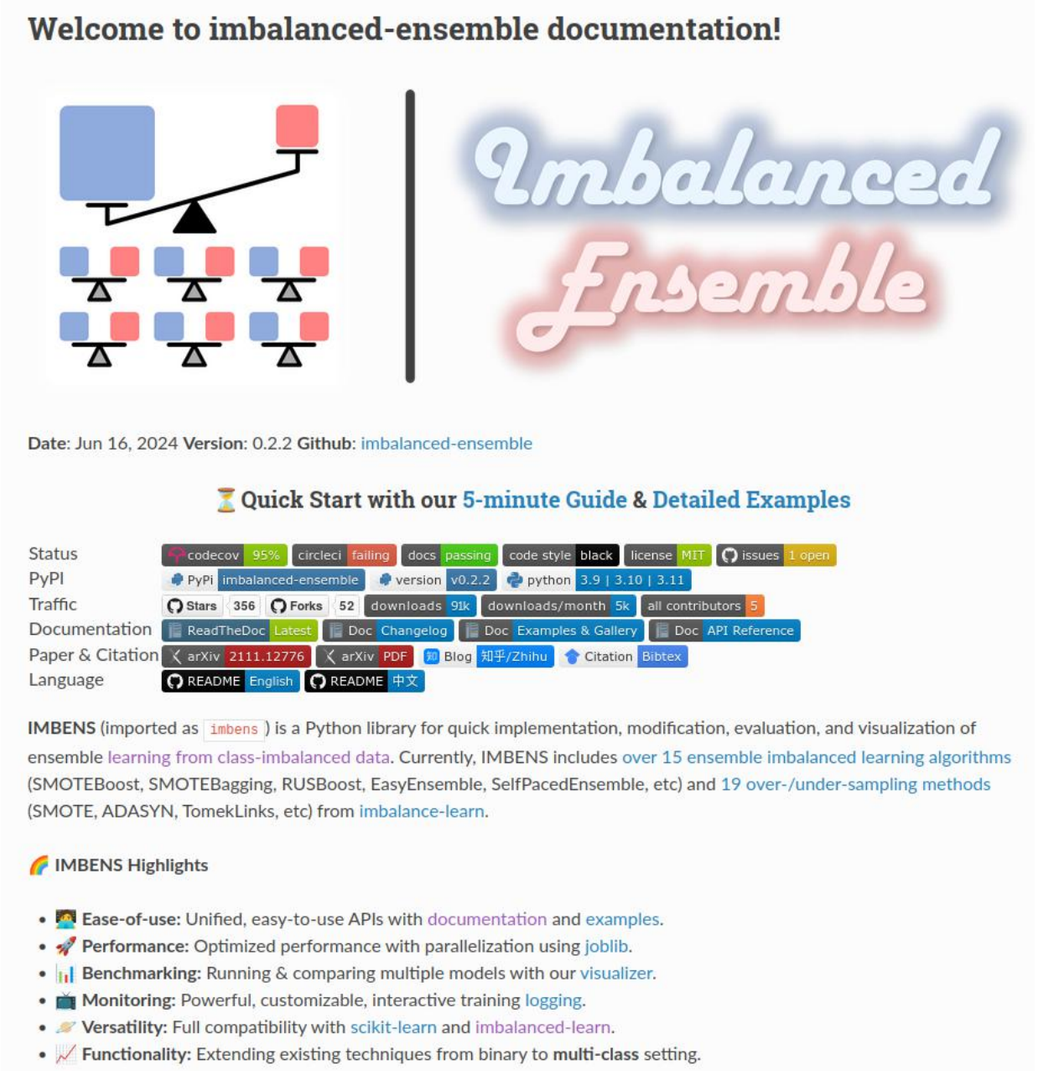
```python
from imbens.ensemble import SelfPacedEnsembleClassifier
from imbens.datasets import generate_imbalance_data
from imbens.utils import evaluate_print
from imbens.visualizer import ImbalancedEnsembleVisualizer

X_train, X_test, y_train, y_test = generate_imbalance_data(
    n_samples=200, weights=[.9,.1], test_size=.5)

clf = SelfPacedEnsembleClassifier()                    # initialize ensemble
clf.fit(X_train, y_train)
y_test_pred = clf.predict(X_test)                      # predict labels
y_test_proba = clf.predict_proba(X_test)               # predict probabilities

evaluate_print(y_test, y_test_pred, "SPE")             # performance evaluation
# SPE balanced Acc: 0.972 | macro Fscore: 0.886 | macro Gmean: 0.972

visualizer = ImbalancedEnsembleVisualizer()            # initialize visualizer
visualizer.fit({'SPE': clf})
visualizer.performance_lineplot()                      # performance visualization
visualizer.confusion_matrix_heatmap()                  # prediction visualization
```

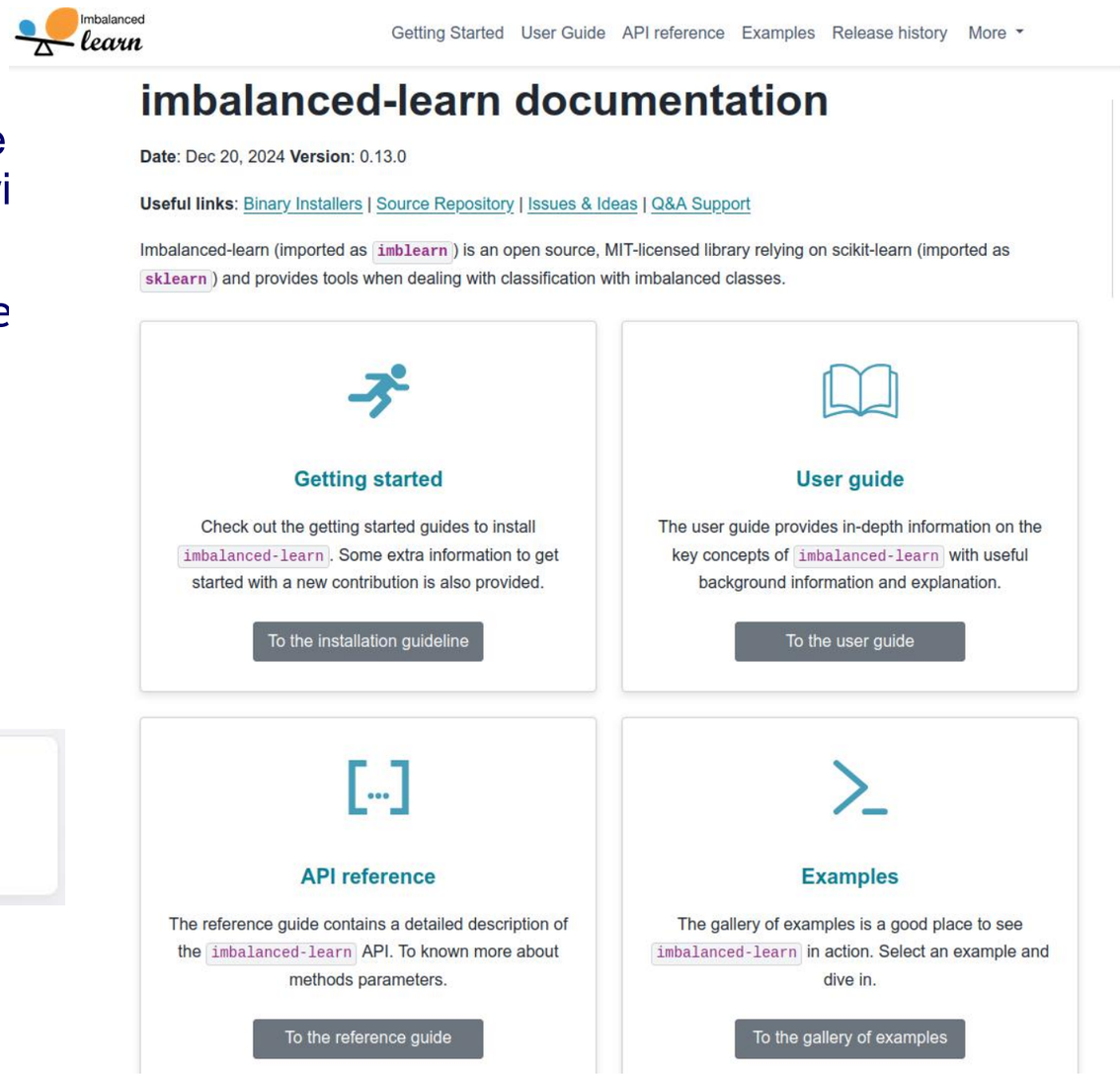


Imbalanced Learning Frameworks and Libraries

https://imbalanced-learn.org/stable/

- `imbalanced-learn` is a python package offering a numbe of re-sampling techniques commonly used in datasets showi strong between-class imbalance.
- Includes 21 different re-sampling techniques, including ove sampling, under-sampling and hybrid ones (e.g., SMOTE, ADASYN, TomekLinks, NearMiss, OneSideSelection, SMOTETomek, ...)
- `pip install imbalanced-learn`

```
from imblearn.over_sampling import RandomOverSampler
ros = RandomOverSampler()
X_resampled, y_resampled = ros.fit_resample(X, y)
```

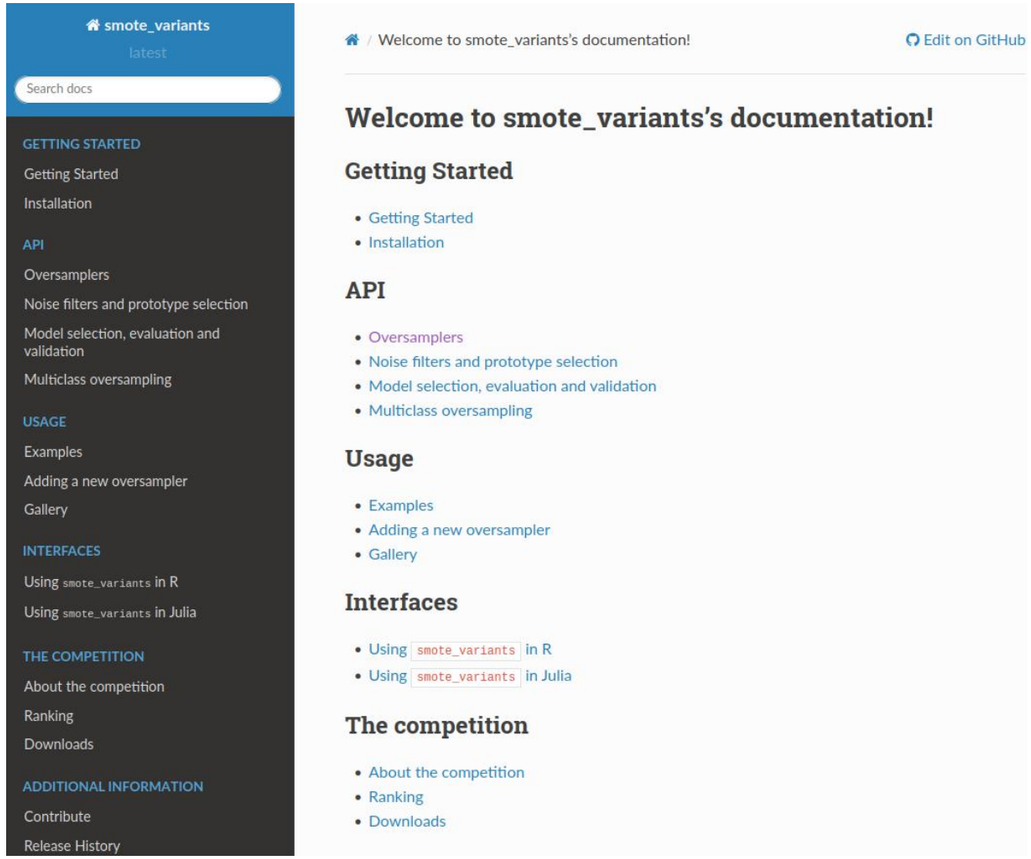


Imbalanced Learning Frameworks and Libraries

https://smote-variants.readthedocs.io/en/latest/index.html

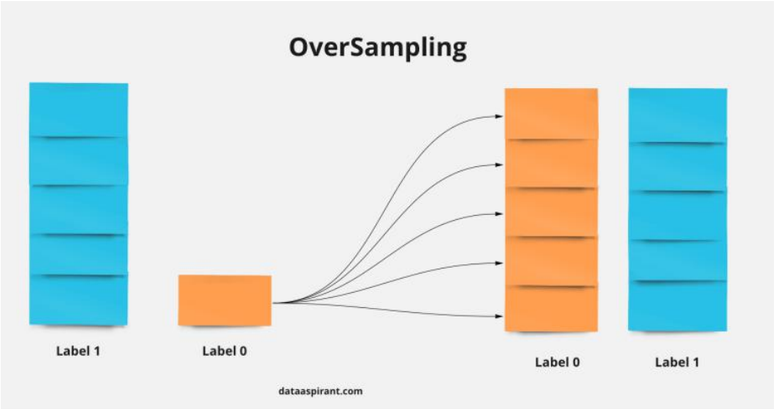
- `smote_variants` proposes a collection of 85 minority over-sampling techniques for imbalanced learning with multi-class oversampling and model selection features (All writen in Python, also support R and Julia).
- `pip install smote_variants`

```
oversampler= smote_variants.SMOTE_ENN()
X_samp, y_samp= oversampler.sample(X, y)
```



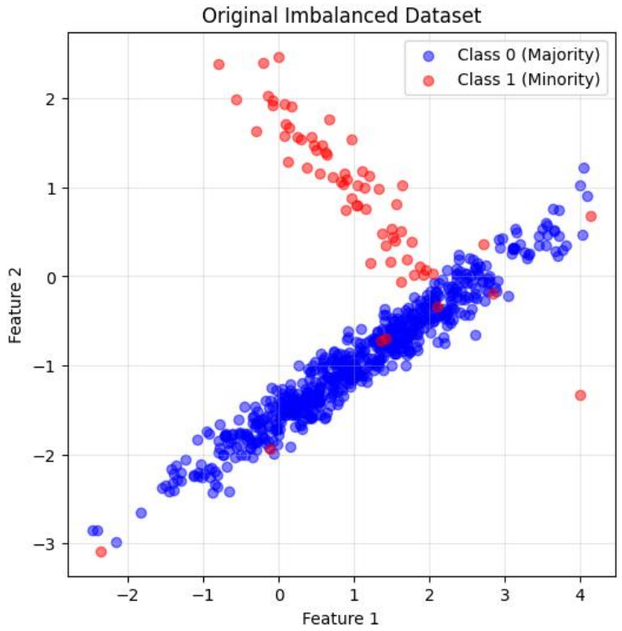
Data-Level - Random Over-sampling

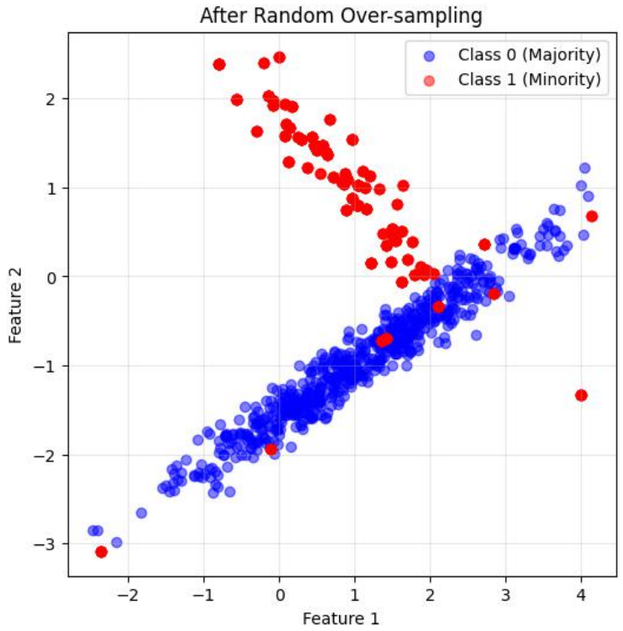
- Randomly duplicates examples from the minority class to match the size of the majority class.
- Preserves all majority class data.
- Works well with small datasets.
- Risk of overfitting the model due to repeated samples.
- Lack of generalization beacause doesn't add any new information—just duplicates.




```python
X, y = make_classification(
        n_samples=1000,
        n_features=2,
        n_informative=2,
        n_redundant=0,
        n_clusters_per_class=1,
        weights=(0.9, 0.1),
        random_state=42
    )
```

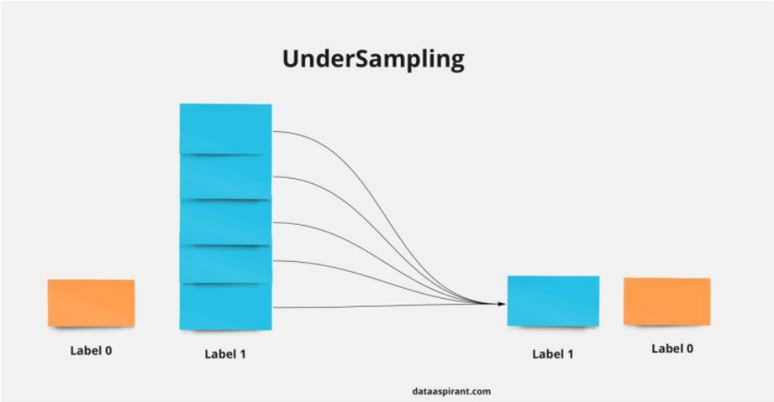
```python
from imblearn.over_sampling import RandomOverSampler
ros = RandomOverSampler(random_state=42)
X_ros, y_ros = ros.fit_resample(X_train, y_train)
plot_data(X_ros, y_ros, "After Random Over-sampling")
```



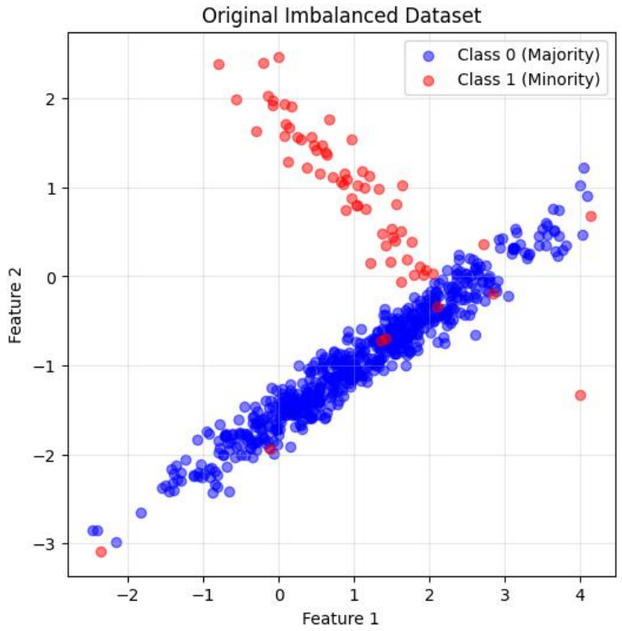


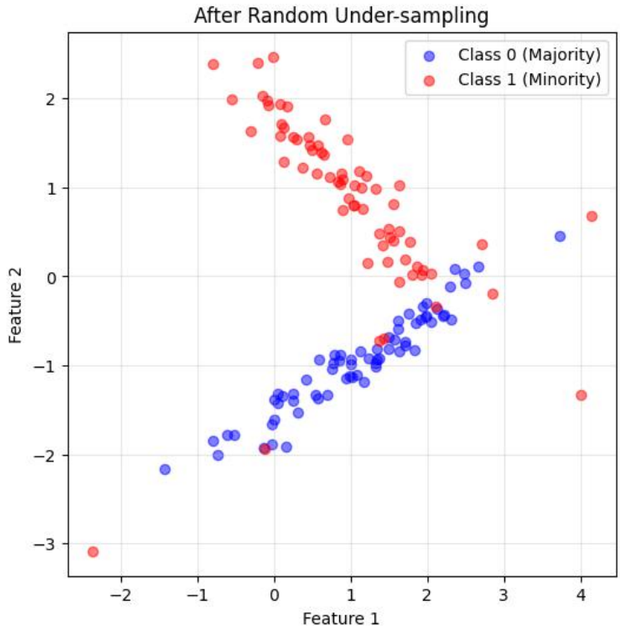
Data-Level - Random Under-sampling

- Randomly removes examples from the majority class to balance with the minority class.
- Reduces training time, when the ***majority class is overwhelmingly large***.
- When the majority of features are categorical, and ***generating good synthetic categorical data can be tricky***.
- Risk of losing important information from the majority class.
- Can lead to underfitting if too much data is discarded.



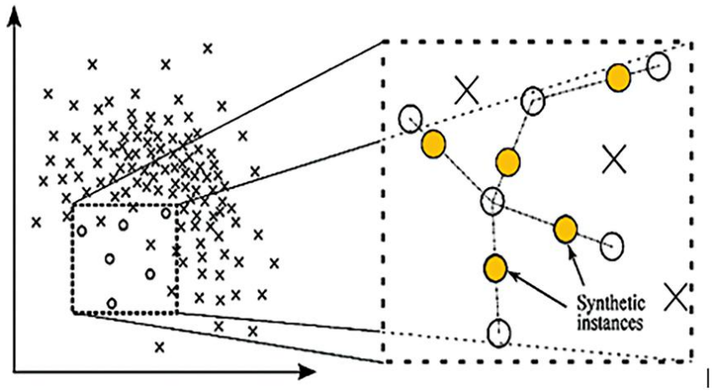

```python
from imblearn.under_sampling import RandomUnderSampler
rus = RandomUnderSampler(random_state=42)
X_rus, y_rus = rus.fit_resample(X_train, y_train)
plot_data(X_rus, y_rus, "After Random Under-sampling")
```



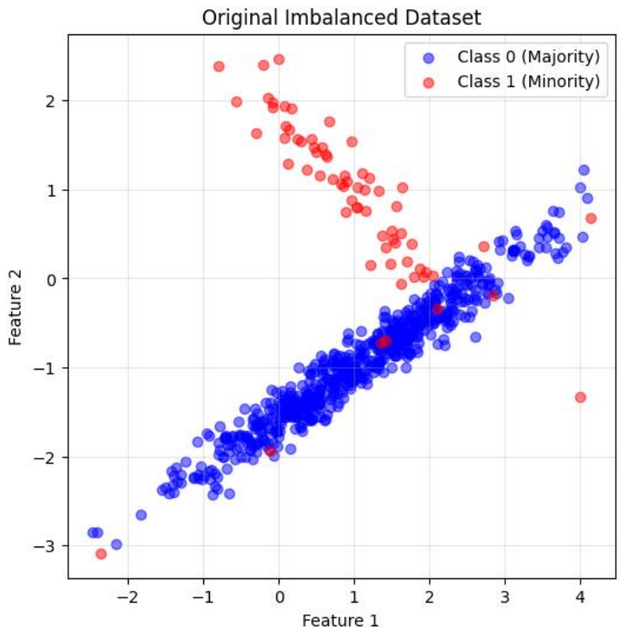


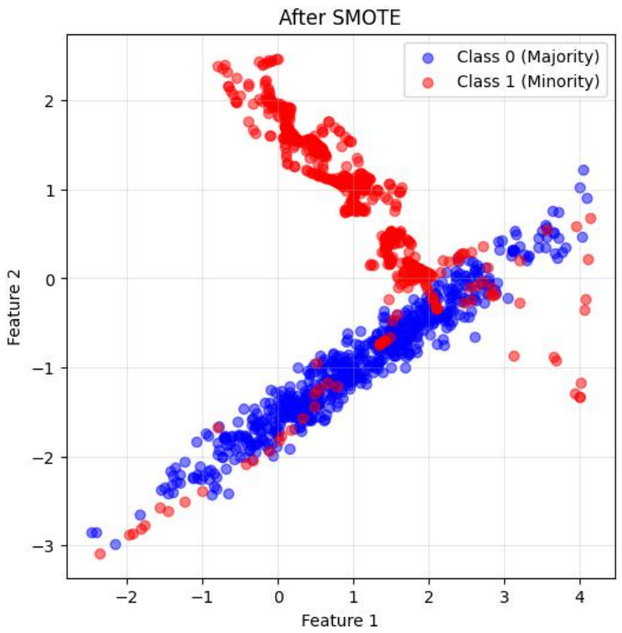
Data-Level - SMOTE

- This method synthesizes samples of minority class as convex (~linear) combinations of existing points and their nearest neighbors of same class.
- Adds new, diverse synthetic examples—less overfitting than random over-sampling. Widely supported and well-researched.
- Risk of creating noisy or overlapping samples, especially if classes are not well-separated.
- Doesn't consider majority class distribution (can lead to over-generalization).




```python
from imblearn.over_sampling import SMOTE
smote = SMOTE(random_state=42)
X_smote, y_smote = smote.fit_resample(X_train,y_train)
plot_data(X_smote, y_smote, "After SMOTE")
```

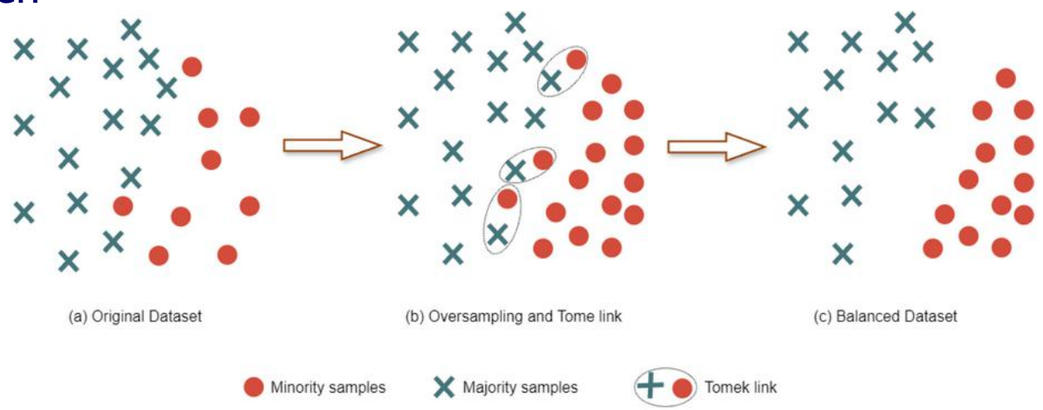




Consider using SMOTE-NC or SMOTE-ENC when you have Numerical and Categorical data.

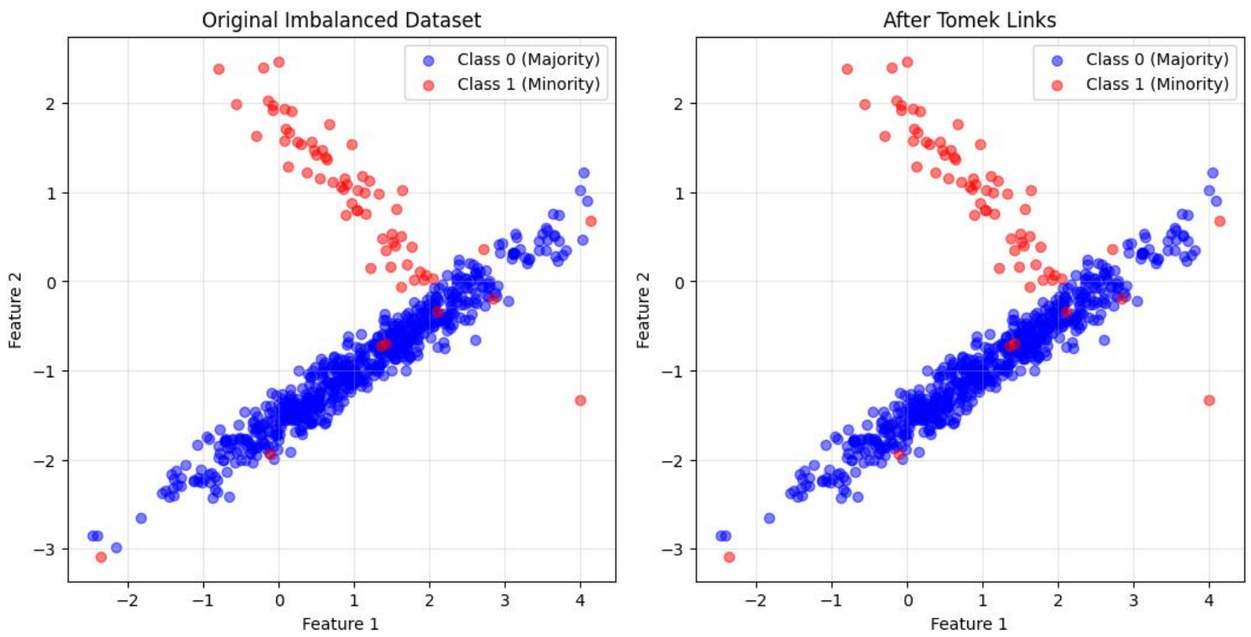
Data-Level - Tomek Links

- Considering a pair of examples (one from each class) that are each other's nearest neighbor and belong to opposite classes. We can remove the majority class examples in each pair to cleans the decision boundary.
- Clean overlapping regions between classes.
- Doesn’t generate new data, only removes samples.
- Ineffective on highly imbalanced datasets alone.



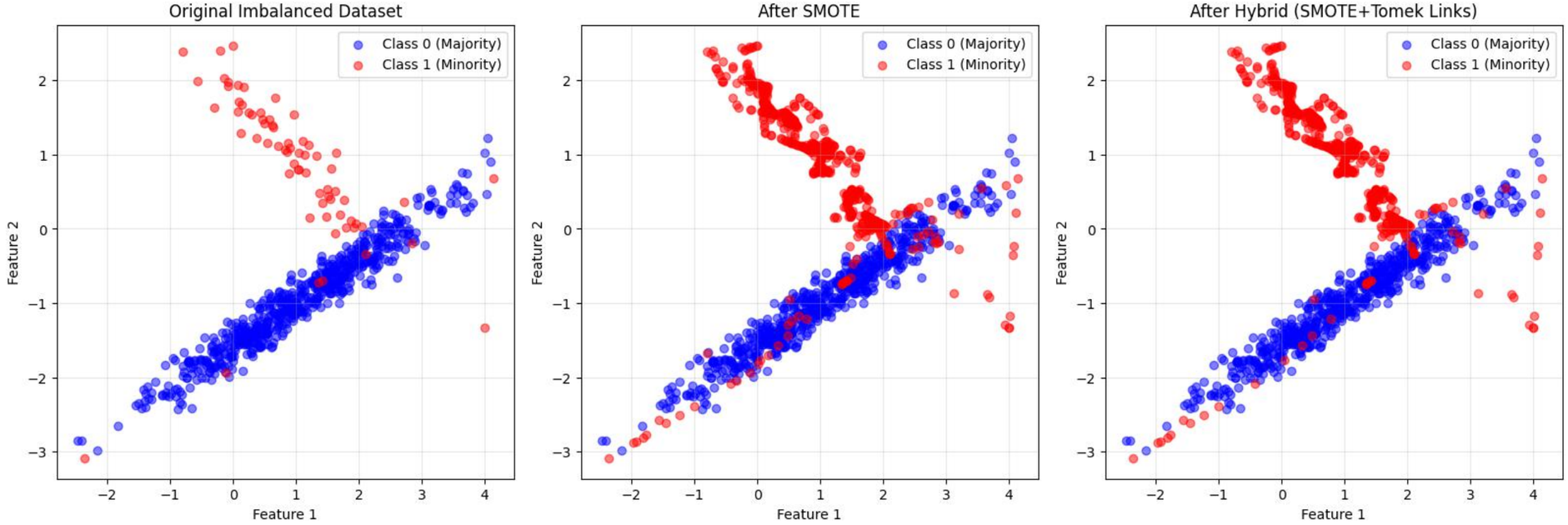

```python
from imblearn.under_sampling import TomekLinks
tl = TomekLinks(sampling_strategy='auto')
X_tl, y_tl = tl.fit_resample(X_train, y_train)
plot_data(X_tl, y_tl, "After Tomek Links")
```

`'minority'`: resample only the minority class;
`'not minority'`: resample all classes but the minority class;
`'not majority'`: resample all classes but the majority class;
`'all'`: resample all classes;
`'auto'`: equivalent to `'not majority'`.



Data-Level - Hybrid : SMOTE + Tomek Links ~= SMOTETomek

- Over-sampling using SMOTE and cleaning using Tomek links.
- Improves both class balance and quality of data.
- Helps model learn better patterns without overfitting to noise.



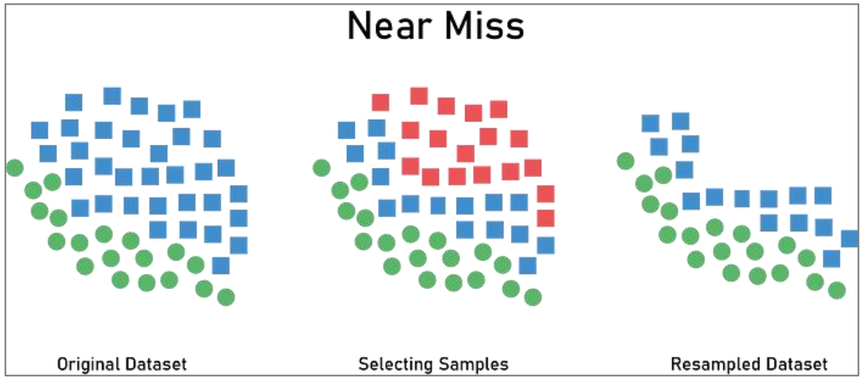

```python
from imblearn.over_sampling import SMOTE
from imblearn.under_sampling import TomekLinks
smote = SMOTE(random_state=42)
tl = TomekLinks(sampling_strategy="all")
X_smote, y_smote = smote.fit_resample(X_train, y_train)
X_smote_tl, y_smote_tl = tl.fit_resample(X_smote, y_smote)
plot_data(X_smote_tl, y_smote_tl, "After Hybrid (SMOTE+Tomek Links)")
```

Similar results by using SMOTETomek

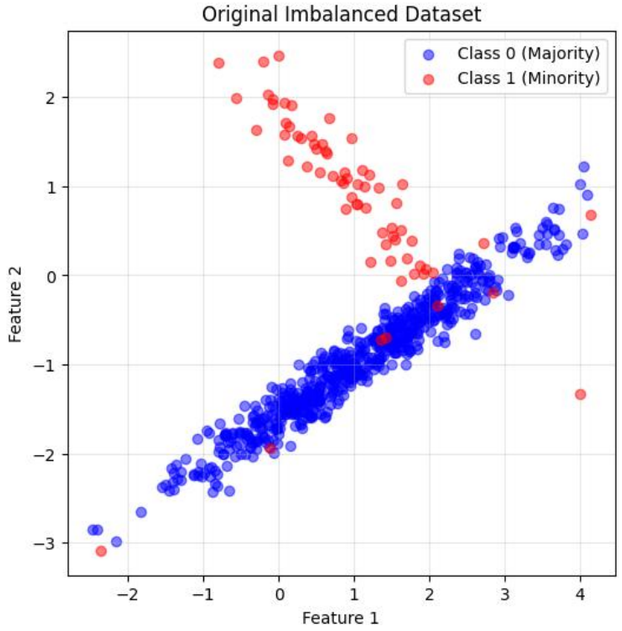
```python
from imblearn.combine import SMOTETomek
smt = SMOTETomek(random_state=42)
X_stom, y_stom = smt.fit_resample(X, y)
plot_data(X_stom, y_stom, "After SMOTETomek")
```

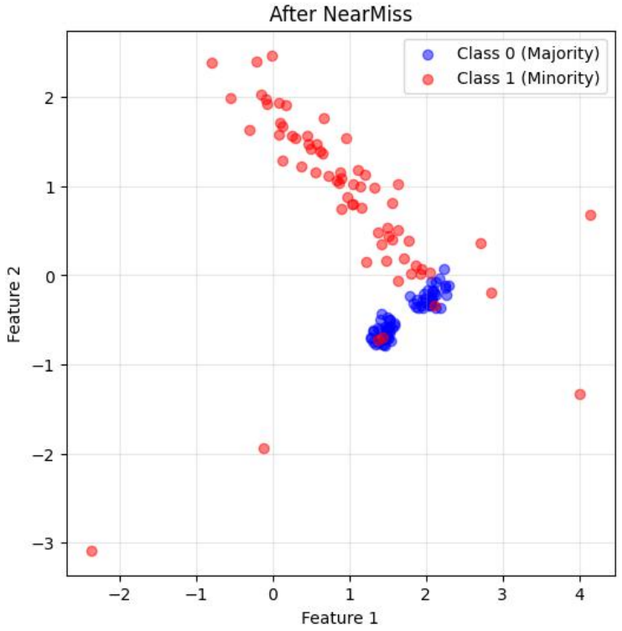
Data-Level - NearMiss

- An under-sampling technique that selects majority class samples that are closest to the minority class using distance-based rules.
- Preserves borderline samples that are most informative.
- Can lead to overlap and noise if not tuned properly.




```python
from imblearn.under_sampling import NearMiss
nm = NearMiss(version=1)  # version=1, 2, or 3
X_nm, y_nm = nm.fit_resample(X_train, y_train)
plot_data(X_nm, y_nm, "After NearMiss")
```





Algorithm-Level

```python
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(
        n_estimators=100,
        class_weight={0:1, 1:10}
    )
```

Class Weighting

- Automatically adjust weights inversely proportional to class frequencies.
- How it's done:
- Many tree-based models and neural networks accept a weight vector for classes.
- Useful in: Decision Trees, Random Forests, Gradient Boosting (like XGBoost, LightGBM), SVMs.

Example: `class_weight={0: 1, 1: 5}` 5x weight for minority class 1.

Cost-Sensitive Learning

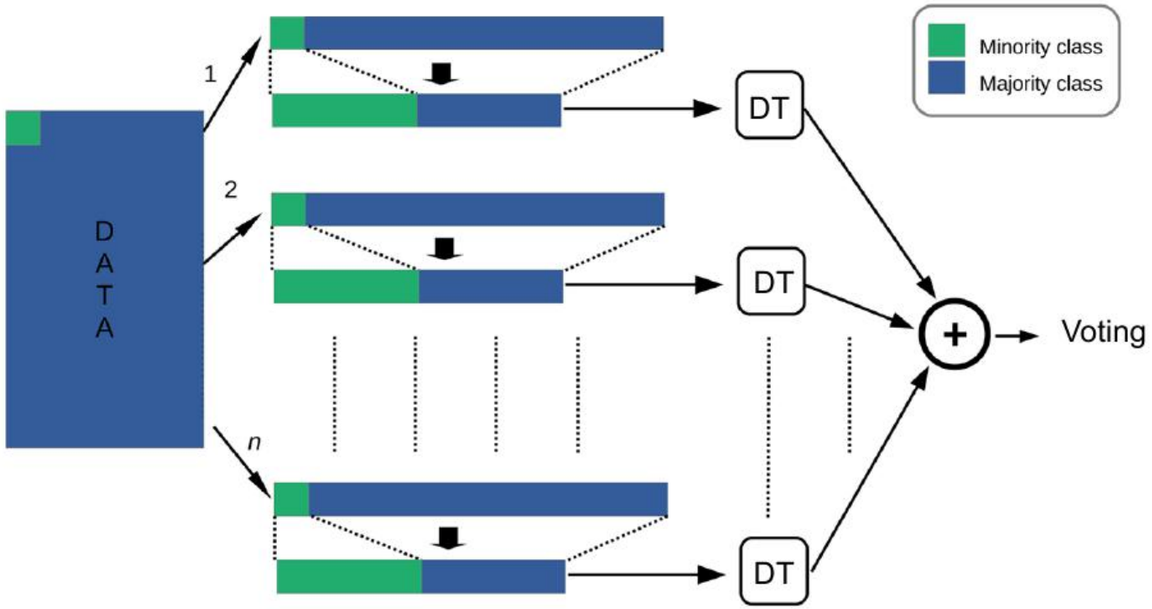
- Assign higher penalties (costs) for misclassifying the minority class.
- How it's done:
- Modify the loss function to include class-specific weights.
- Algorithms like logistic regression, SVM, and neural networks can include these costs.
- Example: `class_weight='balanced'` in scikit-learn classifiers.

Decision Threshold Adjustment

- By modifying the classification threshold, you can control the trade-off between precision and recall in a binary classification problem.
- Lowering the threshold can increase recall, making the model more sensitive to the minority class, though at the cost of precision.

Ensemble-Based Methods - Balanced Random Forest Classifier

- An ensemble method that internally handles class imbalance during training.
- Differs from a classical random forest by the fact that it will draw a bootstrap* sample from the minority class and sample with replacement the same number of samples from the majority class.




```python
from imblearn.ensemble import BalancedRandomForestClassifier
# Train the Balanced Random Forest model
model = BalancedRandomForestClassifier(
    n_estimators=100,
    class_weight="balanced",
    random_state=42)

model.fit(X_train, y_train)
# Make predictions and evaluate
y_pred = model.predict(X_test)
```

* Bootstrapping is a procedure for estimating the distribution of an estimator by resampling.

Ensemble-Based Methods - EasyEnsemble

- The classifier is an ensemble of **AdaBoost** learners trained on different balanced bootstrap samples.
- The balancing is achieved by random under-sampling.
- Utilizes all majority class instances across different subsets.

```
from imblearn.ensemble import EasyEnsembleClassifier
ensemble = EasyEnsembleClassifier(n_estimators=10)
ensemble.fit(X, y)
```

Evaluation-Level - Precision, Recall & F1-Score

Precision: Measures the accuracy of positive predictions.

- It answers the question, “Of all the items the model labeled as positive, how many were actually positive?”

Recall (Sensitivity): Measures the model’s ability to find all the positive instances.

- It answers the question, “Of all the actual positives, how many did the model correctly identify?”

F1 Score: The harmonic mean of precision and recall.

- It balances the two metrics into a single number, making it especially useful when precision and recall are in trade-off.

Precision, Recall, and F1 Score Formulas

1. **Precision:**

$$\text{Precision} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Positives (FP)}}$$

- Focuses on the correctness of positive predictions.

2. **Recall:**

$$\text{Recall} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Negatives (FN)}}$$

- Measures how well the model captures all positive instances.

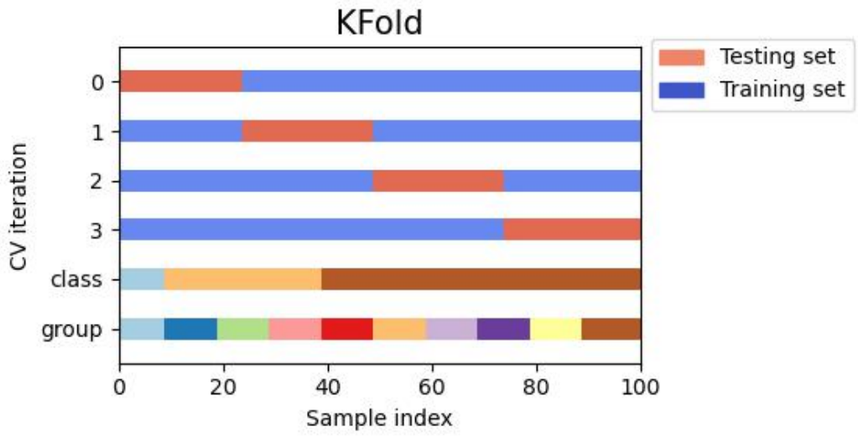
3. **F1 Score:**

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- A single metric that balances precision and recall.

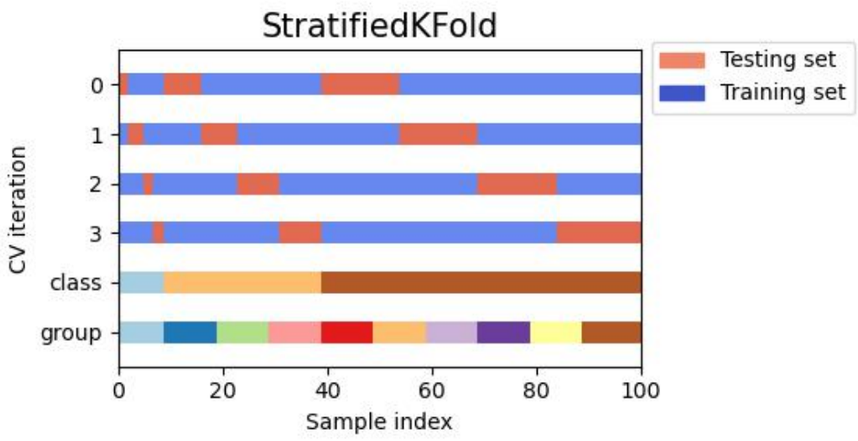
Evaluation-Level - Stratified Cross-Validation

- In the case when imbalanced dataset (e.g., 90% class A and 10% class B), then in a **regular K-Fold**, you might end up with folds that contain only class A or very few examples of class B, which leads to:
- Poor training (model doesn't see enough of class B)
- Unreliable validation (metrics fluctuate a lot)



- **Stratified K-Fold** fixes that by ensuring that each fold maintains approximately the same class distribution as the original dataset.

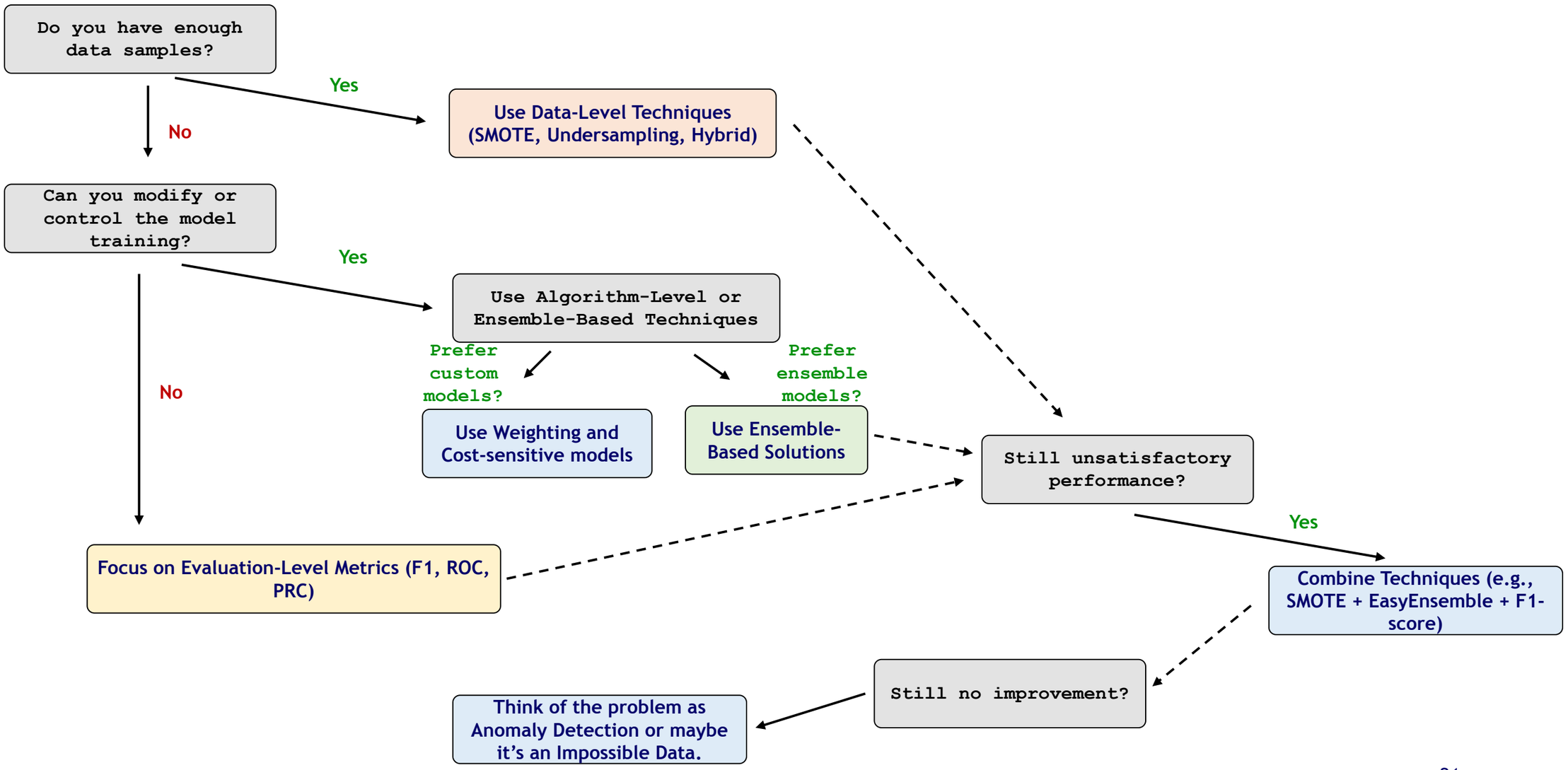
```python
from sklearn.model_selection import StratifiedKFold
skf = StratifiedKFold(n_splits=2)
skf.get_n_splits(X, y)
for i, (train_index, test_index) in enumerate(skf.split(X, y)):
    print(f"Fold {i}:")
    print(f"  Train: index={train_index}")
    print(f"  Test:  index={test_index}")
```



Approaches for Handling Imbalanced Data

Approach	When to Use	Limitations
Data-Level (Resampling)	You have enough data or can synthesize more You want to balance data before training	Can cause overfitting (oversampling) Loss of information (undersampling) Slower training
Algorithm-Level (Cost-sensitive models)	You can change model training parameters You want to penalize minority class errors	May require model support Needs careful tuning of weights Not all models allow it
Ensemble-Based	You want robust performance You can train multiple models	Computationally expensive May be harder to interpret Needs good validation strategy
Evaluation-Level (Better metrics)	You can't modify the model or data You want to assess true performance	Does not improve model performance Needs domain knowledge for metric selection

Decision Flowchart: What Should I Do With Imbalanced Data?



References

The impact of data difficulty factors on classification of imbalanced and concept drifting data streams
https://neptune.ai/blog/how-to-deal-with-imbalanced-classification-and-regression-data
A Review on Solution to Class Imbalance Problem: Undersampling Approaches
https://medium.com/@haataa/fighting-imbalance-data-set-with-code-examples-f2a3880700a6
https://machinelearningmastery.com/5-effective-ways-to-handle-imbalanced-data-in-machine-learning/
https://dataaspirant.com/handle-imbalanced-data-machine-learning/
Imbalance-Aware Machine Learning for Predicting Rare and Common Disease-Associated Non- Coding Variants
smote-enc: A novel SMOTE-based method to generate syn- thetic data for nominal and continuous features
Machine Learning and Integrative Analysis of Biomedical Big Data
A Systematic Review on Imbalanced Data Challenges in Machine Learning: Applications and Solutions
A review on classification of imbalanced data for wireless sensor networks